

Проект «Эмулятор микрокалькулятора «Электроника МК-61s mini»

https://github.com/UN7FGO/MK61S_MINI

Черновик инструкции по TinyBASIC

Версия черновика: 07.07.2026

Этот документ описывает предлагаемую реализацию TinyBASIC для МК61s mini. Это не описание готовой функции, а рабочая спецификация перед разработкой.

TinyBASIC должен быть отдельным режимом со своим меню, своим типом сохраненных программ и тем же общим построчным редактором, который используется FOCAL. Синтаксис основан на Tiny BASIC / TB51: целые номера строк, несколько команд в одной строке через двоеточие, короткие ключевые слова, LET, PRINT, INPUT, IF, GOTO, GOSUB, FOR и NEXT.

При этом числовая семантика должна быть калькуляторной, а не 16-битной целочисленной. TinyBASIC для МК61s mini - это не точная эмуляция TB51, а компактный BASIC с синтаксисом Tiny BASIC и математикой МК-61.

Основные решения

- У TinyBASIC отдельное меню и отдельный список программ.
- Программа состоит из строк с целыми номерами 1...32767.
- Одна строка может содержать несколько команд, разделенных :.
- Синтаксис программы всегда латинский: операторы, функции и переменные записываются английскими именами даже в русском режиме интерфейса.
- В русском режиме локализируются меню, подсказки, сообщения и ошибки.
- Числа, арифметика, тригонометрия и случайные числа используют калькуляторную семантику МК-61, а не 16-битные integer-правила TB51.
- Деление / является обычным вещественным делением: 14/5 дает 2.8.
- Сравнения возвращают 1 или 0.
- IF считает истинным любое ненулевое значение выражения.
- LET можно писать явно, но можно опускать: A=10 эквивалентно LET A=10.
- THEN в IF можно писать явно, но можно опускать.
- Ключевые слова можно сокращать по правилам Tiny BASIC: P. значит PRINT, G. значит GOTO, GOS. значит GOSUB.
- Одно-клавишная вставка операторов, как в FOCAL, в TinyBASIC не используется.
- Редактор TinyBASIC использует общий код редактора, но языковые hooks не вставляют операторы по одной клавише.
- TinyBASIC включен по умолчанию и должен отключаться условной компиляцией через MK61_ENABLE_TINYBASIC.
- Внутренняя сигнатура сохраненной программы: B2.
- Файловое расширение для virtual FAT: TBI.
- Исходником считается обычный текст TinyBASIC. Компилированное представление может быть RAM-кэшем, но не источником истины.

Отличия от исторического TB51

TinyBASIC берет у TB51 синтаксис, но не все поведение.

Сохраняется:

- целые номера строк;
- команды через ;;
- LET, PRINT, INPUT, IF, GOTO, GOSUB, RETURN, FOR, NEXT, END;
- сокращения с точкой;
- опциональные LET и THEN;
- переменные A...Z;
- строки в одинарных или двойных кавычках в PRINT и INPUT.

Изменяется:

- числа не ограничены диапазоном -32767...32767;
- арифметика не целочисленная;
- / не отбрасывает дробную часть;
- логические операции работают как булевы операции над числами, а не как побитовые операции над 16-битными словами;
- IF проверяет ненулевое значение, а не нечетность;
- RND () вызывает калькуляторный генератор случайных чисел;
- ошибки должны быть понятны пользователю MK61s mini, но короткие английские формы WHAT?, HOW?, SORRY можно сохранить как канонические.

Не переносится из TB51 в первую версию:

- 8051-мониторные команды DBYTE, RBIT, XBYTE, CBYTE, CALL;
- команды выбора памяти RAM, ROM, PROM;
- режимы HEX и DECIMAL;
- RESET как команда языка;
- автобауд, serial console и аппаратные особенности 8051.

Строки и номера

Строка начинается с десятичного номера:

```
10 PRINT "HELLO"  
20 END
```

Номер строки:

- целое десятичное число;
- диапазон 1...32767;
- хранится как целый адрес строки, а не как вещественное число;
- отделяется от команд одним или несколькими пробелами.

Рекомендуемый шаг нумерации - 10:

```
10 INPUT "A", A  
20 INPUT "B", B  
30 PRINT A+B  
40 END
```

Так между строками остается место для вставки:

```
10 INPUT A  
15 IF A<0 PRINT "NEG"
```

```
20 PRINT A
```

Порядок выполнения программы определяется числовым порядком строк. Исходный текст желательно хранить и показывать в отсортированном виде. Если в исходнике встречаются две строки с одинаковым номером, первая версия должна считать это ошибкой WHAT?, а не молча выбирать одну из них.

Пустые строки в исходнике игнорируются.

Команды в строке

В одной строке может быть несколько команд:

```
10 A=1:B=2:PRINT A+B
```

Команды разделяются двоеточием :.

Некоторые команды должны быть последними в строке:

- GOTO;
- GOSUB;
- RETURN;
- FOR;
- END;
- STOP;
- REM.

Это правило оставляет выполнение простым и близким к Tiny BASIC. Например:

```
10 IF A>0 GOTO 100  
20 PRINT "A<=0"
```

А так писать не нужно:

```
10 GOTO 100:PRINT "UNREACHABLE"
```

Редактор программы

Редактор TinyBASIC должен быть построчным и использовать общий модуль редактора. Поведение окна, курсора, SMS-ввода и навигации должно совпадать с FOCAL.

Редактор показывает окно листинга вокруг текущей строки:

- на LCD1602 и других двухстрочных дисплеях - 2 строки программы;
- на графическом дисплее - 4 строки программы.

Текущая строка отмечается маркером > или инверсией:

```
10 INPUT A  
>20 PRINT A*A  
30 END
```

На двухстрочном дисплее показываются текущая строка и соседняя строка:

```
>20 PRINT A*A  
30 END
```

Если строка длиннее ширины дисплея, строка прокручивается горизонтально вокруг текстового курсора. Номер строки желательно оставлять видимым, пока это возможно.

Навигация в редакторе

Навигация такая же, как в FOCAL:

Действие	Клавиша
Курсор на символ влево	стрелка влево
Курсор на символ вправо	стрелка вправо
Предыдущая строка программы	ШГ<-
Следующая строка программы	ШГ->
Принять строку и перейти к следующей	OK / Enter
Сохранить программу и выйти	MENU/ESC
Удалить символ слева	F + стрелка влево
Очистить текущую строку	Cx
Удалить пустую текущую строку	Cx на пустой строке

OK в TinyBASIC-редакторе вставляет новую строку. Двоеточие : как разделитель команд вводится отдельной комбинацией, чтобы не конфликтовать с переходом на следующую строку.

Ввод операторов

В отличие от FOCAL, TinyBASIC не должен вставлять операторы по одной клавише. Клавиши, которые в FOCAL вставляют ASK, BRANCH, PRINT, SET и другие операторы, в TinyBASIC не должны делать этого.

Команды вводятся как обычный текст:

```
PRINT
INPUT
GOTO
GOSUB
RETURN
FOR
NEXT
END
```

Чтобы набор оставался коротким, нужно использовать сокращения Tiny BASIC:

Полная форма	Краткая форма
PRINT	P.
INPUT	IN.
IF	I.
GOTO	G.
GOSUB	GOS.
RETURN	R.
FOR	F.
NEXT	N.
END	E.
REM	REM
LET	можно опустить
THEN	можно опустить

Например:

```
10 P. "HELLO"
20 E.
```

эквивалентно:

```
10 PRINT "HELLO"
```

Аббревиатура должна заканчиваться точкой, кроме тех слов, где краткая форма совпадает с полным словом (REM, NEW в direct-режиме, если он будет добавлен позже).

Ввод символов

TinyBASIC использует тот же общий принцип префиксов, что FOCAL:

- обычные 0...9 вводят цифры;
- K + SMS-клавиша включает SMS-ввод латинских букв или пробела;
- F + цифровая клавиша вводит спецсимвол;
- K включает ввод дополнительного символа на одно следующее нажатие;
- после ввода одного символа префикс сбрасывается.

SMS-ввод букв:

Клавиша	Цикл символов
K + 7	пробел
K + 8	A, B, C
K + 9	D, E, F
K + 4	G, H, I
K + 5	J, K, L
K + 6	M, N, O
K + 1	P, Q, R, S
K + 2	T, U, V
K + 3	W, X, Y, Z

После K + буквенной SMS-клавиши повтор той же цифры уже без K заменяет последнюю введенную букву на следующую из цикла. Другая буквенная SMS-клавиша начинает следующую букву. В SMS-режиме курсор показывается иначе, как _.

Из SMS-режима выводят:

- пауза ввода;
- пробел ПП, он же вводит пробел;
- клавиша 7, она завершает SMS-режим и вводит пробел;
- клавиша 0, она только завершает SMS-режим.

Цифровая K + 0 не начинает SMS-ввод.

Спецсимволы:

Клавиша	Символ
F + 0	!
F + 1	@
F + 2	#
F + 3	\$
F + 4	%
F + 5	^
F + 6	&

Клавиша	Символ
F + 7	*
F + 8	(
F + 9	)

Основные символы TinyBASIC:

Символ	Как нажать
пробел	ПП или SMS-клавиша 7
цифры	цифровые клавиши
.	клавиша десятичной точки
+, -, *, /	соответствующие арифметические клавиши
=	К + +
<	К + ШГ<-
>	К + -
(	К + стрелка влево или F + 8
)	К + стрелка вправо или F + 9
"	К + X<->Y
'	К + .
,	К + ПП
:	К + ОК
;	К + В/О
^	К + ВП или F + 5
!	F + 0

Составные операторы сравнения набираются из отдельных символов:

```
<>
<=
>=
```

Локализация

Текущий язык интерфейса влияет на меню, подсказки, сообщения редактора, ввод числа и ошибки. В русском режиме пользователь должен видеть русские сообщения.

Исходный текст программы при этом остается латинским и стабильным:

```
10 INPUT "A", A
20 PRINT SIN(A)
30 END
```

Не должно быть русских вариантов операторов или функций. Например, в русском режиме пользователь вводит PRINT, INPUT, GOTO, SIN, RND, а не русские ключевые слова.

Условная компиляция

TinyBASIC должен отключаться на этапе сборки. Флаг:

```
#ifndef MK61_ENABLE_TINYBASIC
#define MK61_ENABLE_TINYBASIC 1
#endif
```

Значение по умолчанию - 1, то есть TinyBASIC включен.

Если MK61_ENABLE_TINYBASIC равен 0, из прошивки должны исключаться:

- пункт меню TinyBASIC;
- редактор TinyBASIC;
- парсер и компилятор TinyBASIC;
- runtime TinyBASIC;
- хранилище программ TinyBASIC;
- тестовые или сервисные входы, которые доступны только для TinyBASIC.

Публичные функции TinyBASIC в заголовке должны иметь пустые inline-заглушки, чтобы остальной код мог собираться без дополнительных `#ifdef` в местах вызова.

Общий редактор не должен иметь отдельного пользовательского флага включения. Он является библиотекой: если включен FOCAL или TinyBASIC, язык подключает редактор; если оба языка отключены, редактор не должен попадать в прошивку.

Хранение программ

TinyBASIC-программа должна храниться отдельно от обычных программ МК-61, старого встроенного BASIC и FOCAL.

Решение первой версии:

```
type/signature = B2
extension      = TBI
name
source_length
source_text
checksum
```

Сигнатура B2 важна, потому что старый встроенный BASIC уже занимает семейство B1 и расширение BAS. TinyBASIC не должен случайно загрузить старую BASIC-программу как TinyBASIC-программу.

Расширение virtual FAT:

```
NAME.TBI
```

Практичная схема работы:

- при открытии редактора программа загружается из постоянного хранилища в RAM;
- редактор работает с RAM-копией;
- при выходе из редактора программа компилируется или проверяется;
- если проверка прошла, исходник записывается обратно в слот;
- при запуске сохраненная программа читается из слота и компилируется в рабочее представление;
- если места не хватает, пользователь получает ошибку SORRY / нет места.

Компилированное или токенизированное представление можно держать в RAM как кэш. Сохраненный текст должен оставаться источником истины. Так программу проще редактировать, переносить через terminal/USB и обновлять между версиями прошивки.

Direct-команды

Исторический Tiny BASIC имел direct-команды RUN, LIST, LIST n и NEW. В МК61s mini основным интерфейсом является меню, поэтому в первой версии эти команды не обязаны быть частью языка программ.

Соответствие меню:

Историческая команда	Действие в МК61s mini
RUN	пункт меню Запуск

Историческая команда	Действие в MK61s mini
LIST	редактор программы
LIST n	переход к строке в редакторе, если будет добавлен
NEW	пункт Новая в выборе программ

Если позже появится терминальный direct-режим TinyBASIC, эти команды можно добавить как команды интерактивной строки. В сохраненной программе они не нужны.

Операторы

Канонические операторы первой версии:

```
REM
LET
PRINT
INPUT
IF
GOTO
GOSUB
RETURN
FOR
NEXT
END
STOP
```

Краткие формы с точкой должны приниматься парсером:

```
P. "HELLO"
IN. A
I. A>0 P. A
G. 100
GOS. 200
R.
F. I=1 TO 10
N. I
E.
```

LET можно опускать:

```
10 LET A=10
20 B=A*2
```

THEN можно опускать:

```
10 IF A>0 THEN PRINT A
20 IF A>0 PRINT A
```

Обе строки выше эквивалентны.

REM

REM начинает комментарий.

```
10 REM SUM OF SQUARES
20 A=3:B=4
30 PRINT A^2+B^2
```

Комментарий продолжается до конца строки. Двоеточия внутри комментария не разделяют команды:

```
10 REM THIS : IS COMMENT
```

REMARK можно принять как полную форму, но каноническая форма для набора на калькуляторе - REM.

LET

LET присваивает значение переменной:

```
10 LET A=234-5*6
20 LET B=A/2
```


LET можно опустить:

```
10 A=234-5*6
20 B=A/2
```

Последовательное присваивание из Tiny BASIC сохраняется:

```
10 LET A=1,2,3
```

Эта строка присваивает:

```
A=1
B=2
C=3
```

Последовательное присваивание начинается с указанной переменной и идет по алфавиту. Если список выходит за Z, это ошибка NOW?.

PRINT

PRINT выводит значения и строки:

```
10 PRINT "HELLO"
20 PRINT A
30 PRINT A*3+1, "ABC", 'CBA'
```

Строки можно заключать в двойные или одинарные кавычки:

```
10 PRINT "TEXT"
20 PRINT 'TEXT'
```

Пустой PRINT выводит перевод строки:

```
10 PRINT
```

Разделитель , переводит вывод к следующей табуляционной позиции. Разделитель ; печатает следующий элемент сразу после предыдущего:

```
10 PRINT "A=", A
20 PRINT "A="; A
```

Если список PRINT заканчивается , или ;, перевод строки после команды не добавляется:

```
10 PRINT "A=";
20 PRINT A
```

INPUT

INPUT читает числа в переменные:

```
10 INPUT A
20 INPUT A,B
```

Перед вводом по умолчанию показывается короткая подсказка:

:

Строки в списке INPUT заменяют подсказку:

```
10 INPUT "A", A
20 INPUT "B", B
```

Можно вводить несколько переменных:

```
10 INPUT "A", A, "B", B
```

Ввод числа должен использовать общий калькуляторный ввод чисел и текущую локализацию интерфейса. В исходном тексте программы десятичная точка всегда пишется как ..

IF

IF проверяет выражение и выполняет оставшуюся часть строки, если выражение ненулевое:

```
10 IF A>B PRINT "A"  
20 IF A=B THEN PRINT "EQ"
```

Если выражение равно нулю, оставшаяся часть строки пропускается, а выполнение продолжается со следующей строкой.

После условия может идти одна команда или несколько команд через :.

```
10 IF A<0 PRINT "NEG":A=-A  
20 PRINT A
```

ELSE в первой версии не поддерживается. Ветвление с альтернативой пишется через GOTO:

```
10 IF A<0 GOTO 100  
20 PRINT "POS"  
30 GOTO 200  
100 PRINT "NEG"  
200 END
```

GOTO

GOTO передает управление строке:

```
10 GOTO 100
```

Адрес может быть выражением:

```
10 GOTO A*10+B
```

Результат выражения должен быть целым номером строки. Если результат не является целым номером или такой строки нет, выполнение останавливается с ошибкой NOW?.

GOTO должен быть последней командой в строке.

GOSUB и RETURN

GOSUB вызывает подпрограмму:

```
10 GOSUB 100  
20 END  
100 PRINT "SUB"  
110 RETURN
```

RETURN возвращает выполнение к команде после последнего GOSUB.

GOSUB может быть вложенным. Глубина стека вызовов должна быть фиксированной и проверяться. Если стек переполнен или RETURN выполнен без GOSUB, это ошибка NOW?.

GOSUB и RETURN должны быть последними командами в строке.

FOR и NEXT

FOR начинает цикл:

```
10 FOR I=1 TO 10  
20 PRINT I  
30 NEXT I
```

Семантика первой версии:

- начальное выражение вычисляется один раз;
- конечное выражение после TO вычисляется один раз при входе в цикл;
- шаг всегда равен 1;
- NEXT I увеличивает переменную I на 1;

- цикл продолжается, пока переменная не превысит сохраненный предел.

FOR должен быть последней командой в строке:

```
10 FOR I=1 TO 10
20 PRINT I
30 NEXT I
```

Так писать не нужно:

```
10 FOR I=1 TO 10:PRINT I:NEXT I
```

NEXT должен указывать ту же переменную, что и ближайший активный FOR. Несовпадение переменной - ошибка HOW?.

STEP в первой версии не поддерживается.

END и STOP

END завершает выполнение программы:

```
10 PRINT "DONE"
20 END
```

STOP делает то же самое:

```
10 STOP
```

END можно не писать в последней строке программы. Если выполнение дошло до конца программы, это считается нормальным завершением.

END и STOP должны быть последними командами в строке.

Переменные

В первой версии есть 26 числовых переменных:

```
A B C ... Z
```

Переменные хранят калькуляторные числа. Начальное значение всех переменных - 0.

Строковых переменных, массивов и пользовательских функций в первой версии нет. Строки используются только как литералы в PRINT, INPUT и REM.

Выражения

Поддерживаются:

- калькуляторные числа с десятичной точкой;
- переменные A...Z;
- PI;
- круглые скобки;
- арифметика +, -, *, /, ^, MOD;
- унарные +, -, NOT;
- сравнения >, <, =, <>, >=, <=;
- логические операции AND, OR, XOR;
- вызовы встроенных функций.

Примеры:

```
A+B
2*PI*R
SIN(A)^2+COS(A)^2
1/(A+B)
```

```
SQRT(A^2+B^2)
(A>B)*100+(A<=B)*200
```

Оператор ^ означает возведение в степень:

X^Y

MOD для калькуляторных чисел определяется так:

$A \text{ MOD } B = A - \text{INT}(A/B) * B$

Деление или MOD на ноль - ошибка NOW?.

Приоритет операций

Базовый порядок вычисления:

- 1 круглые скобки и вызовы функций;
- 2 возведение в степень ^;
- 3 унарные операции +, -, NOT;
- 4 *, /, MOD, AND;
- 5 +, -, OR, XOR;
- 6 сравнения >, <, =, <>, >=, <=.

Внутри одного уровня операции вычисляются слева направо, кроме случаев, которые будут явно зафиксированы тестами для калькуляторной совместимости.

Сравнения возвращают 1 или 0.

Логические операции в TinyBASIC булевы:

- 0 считается ложью;
- любое ненулевое значение считается истиной;
- результат AND, OR, XOR, NOT равен 1 или 0.

Это отличается от TB51, где логические операции были побитовыми над 16-битными целыми числами.

Функции МК-61

TinyBASIC должен использовать тот же канонический набор функций, что FOCAL:

SIN(X)	COS(X)	TG(X)
ASIN(X)	ACOS(X)	ATG(X)
LN(X)	LG(X)	EXP(X)
SQRT(X)	ABS(X)	
INT(X)	FRAC(X)	ROUND(X)
SGN(X)	MAX(X, Y)	
RND()		
RND(X)		
PI		

RND() возвращает результат калькуляторной операции СЧ.

RND(X) - совместимость с Tiny BASIC. Она возвращает случайное целое от 1 до INT(X) включительно.

Если $X < 1$, это ошибка NOW?. Внутри RND(X) используется тот же калькуляторный генератор, что и RND().

Алиасы не поддерживаются:

- нет TAN, только TG;
- нет ATAN, только ATG;
- нет LOG, только LG;

- нет SQR, вместо этого выражение X^2 ;
- нет INV, вместо этого выражение $1/X$;
- нет EXP10, вместо этого выражение 10^X .

Тригонометрический режим

Тригонометрические функции должны использовать текущий тригонометрический режим калькулятора, если это технически возможно.

```
10 INPUT "A", A
20 PRINT SIN(A)
30 END
```

Поведение SIN, COS, TG, ASIN, ACOS, ATG должно совпадать с калькуляторным поведением МК-61 и тестироваться отдельно.

Редакторские макросы выражений

В первой версии TinyBASIC не использует FOCAL-операторные макросы и не вставляет операторы по одной клавише.

Калькуляторные макросы выражений (x^2 , $1/x$, 10^x) можно добавить позже, если они не будут конфликтовать с обычным вводом TinyBASIC. Базовая версия должна оставаться простой: все команды и функции набираются текстом или сокращениями.

Примеры программ

Hello

```
10 PRINT "HELLO"
20 END
```

Короткая форма:

```
10 P. "HELLO"
20 E.
```

Сумма двух чисел

```
10 INPUT "A", A
20 INPUT "B", B
30 PRINT "S="; A+B
40 END
```

Квадраты от 1 до 10

```
10 FOR I=1 TO 10
20 PRINT I, I^2
30 NEXT I
40 END
```

Гипотенуза

```
10 INPUT "A", A
20 INPUT "B", B
30 PRINT SQR(A^2+B^2)
40 END
```

Модуль числа

```
10 INPUT A
20 IF A<0 A=-A
30 PRINT A
40 END
```

Через функцию:

```

10 INPUT A
20 PRINT ABS(A)
30 END

```

Подпрограмма

```

10 A=3
20 GOSUB 100
30 A=4
40 GOSUB 100
50 END
100 PRINT A^2
110 RETURN

```

Выбор строки выражением

```

10 INPUT "N", N
20 GOTO 100+10*N
100 PRINT "ZERO"
110 PRINT "ONE"
120 PRINT "TWO"
130 END

```

Такой прием требует, чтобы вычисленная строка существовала. Если строки нет, будет ошибка HOW?.

Кубик

Через калькуляторный RND():

```

10 D=1+INT(6*RND())
20 PRINT D
30 END

```

Через совместимый RND(X):

```

10 PRINT RND(6)
20 END

```

Ошибки

Канонические короткие ошибки Tiny BASIC:

```

WHAT?
HOW?
SORRY

```

Рекомендуемая семантика:

- WHAT? - синтаксическая ошибка: неизвестная команда, неправильная строка, незакрытая кавычка, неправильное выражение.
- HOW? - команда понятна, но выполнить ее нельзя: деление на ноль, несуществующая строка в GOTO, RETURN без GOSUB, переполнение стека, неправильная переменная в NEXT.
- SORRY - нехватка памяти или места в хранилище.

В русском интерфейсе можно показывать локализованные сообщения, но короткие формы стоит сохранить в отладке и тестах, потому что они компактны и исторически узнаваемы.

Что не входит в первую версию

- 8051-команды DBYTE, RBIT, XBYTE, CBYTE, CALL.
- RAM, ROM, PROM, RESET.
- HEX и DECIMAL.
- Побитовая 16-битная семантика AND, OR, XOR, NOT.
- 16-битное целочисленное переполнение TB51.

- ELSE.
- STEP в цикле FOR.
- DATA, READ, RESTORE.
- массивы;
- строковые переменные;
- пользовательские функции;
- direct-режим через терминал;
- одно-клавишная вставка операторов в редакторе TinyBASIC.

Открытые решения перед реализацией

- 1 Нужно ли редактору автоматически сортировать строки по номеру при сохранении или только компилятору при запуске.
- 2 Должен ли LIST n появиться как отдельное действие редактора или direct- режима.
- 3 Нужно ли разрешить NEXT без переменной как сокращение ближайшего активного цикла.
- 4 Нужно ли добавлять STEP во второй версии или оставить язык ближе к Tiny BASIC.
- 5 Нужно ли показывать ошибки как WHAT?, HOW?, SORRY в пользовательском интерфейсе или локализовать их, оставив английские формы только в тестах.